Process Synchronization 1

Why is Synchronization Needed? 1/4

```
int  Count = 10;
```

Process 1	Process 2
Count++;	Count--;

Count = ? 9, 10 or 11?

Why is Synchronization Needed? 2/4

```
int  Count = 10;
```

Process 1

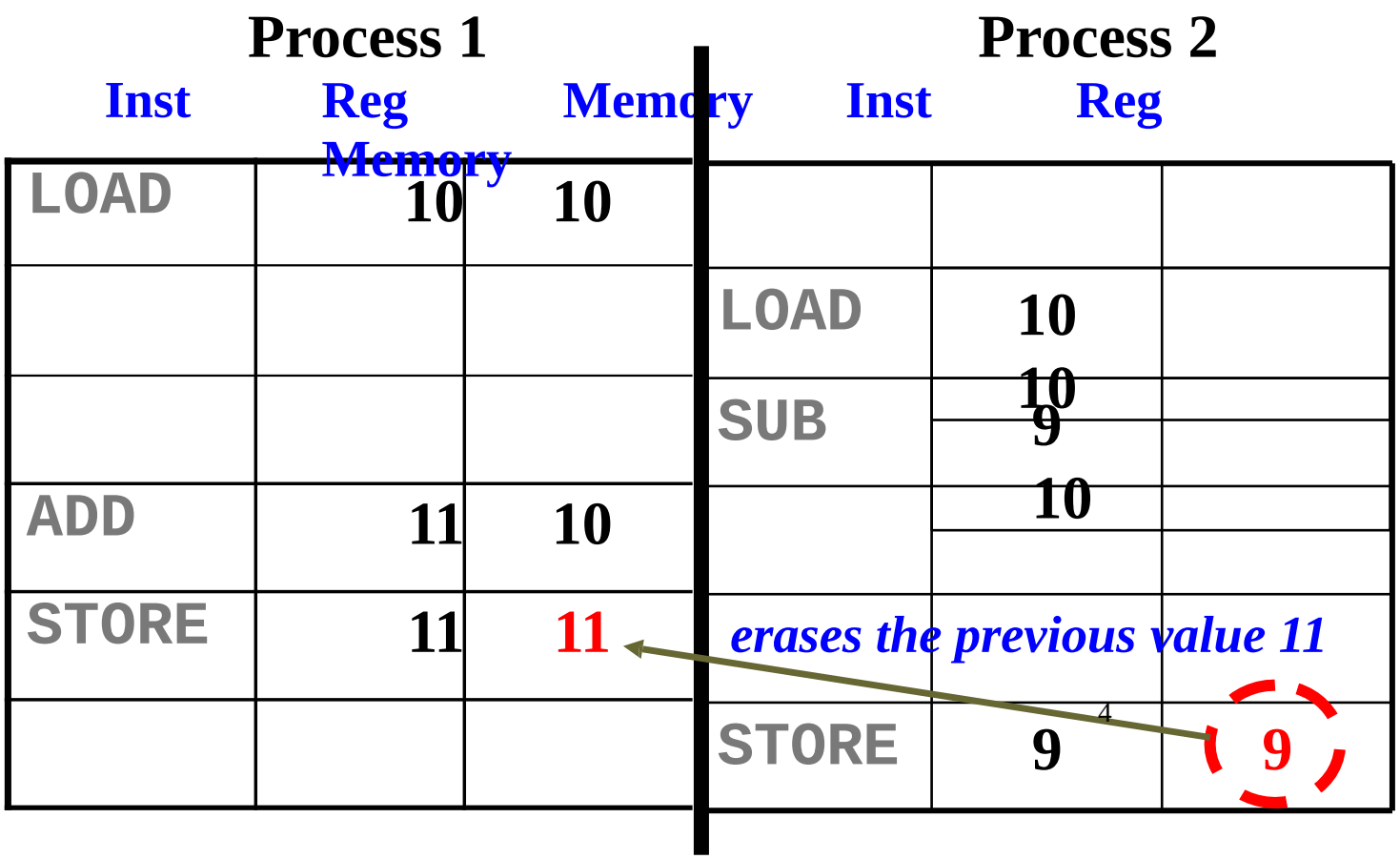
```
LOAD  Reg, Count
ADD   #1
STORE Reg, Count
```

Process 2

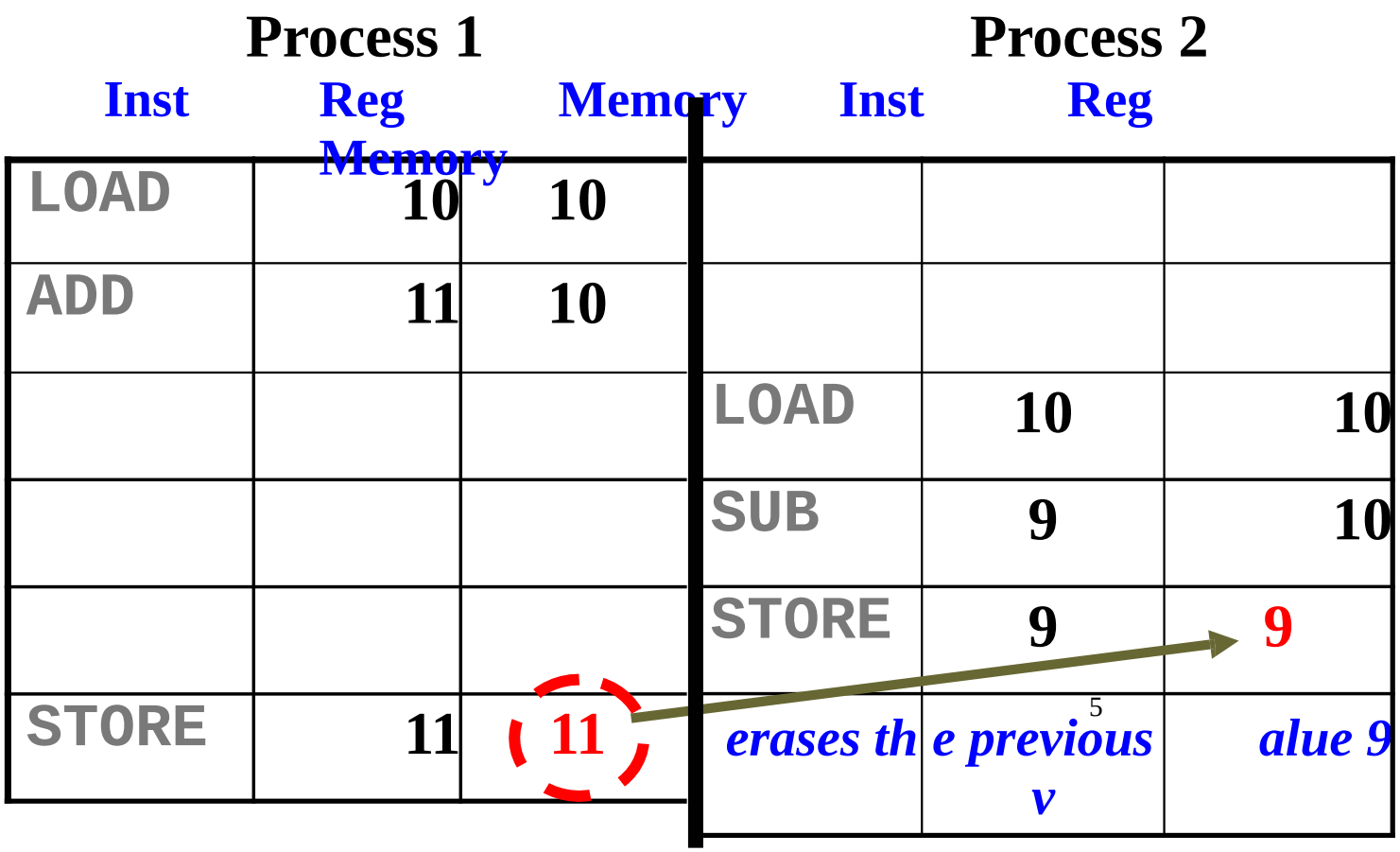
```
LOAD  Reg, Count
SUB   #1
STORE Reg, Count
```

The problem is that the execution flow may be switched in the middle!

Why is Synchronization Needed? 3/4



Why is Synchronization Needed? 4/4

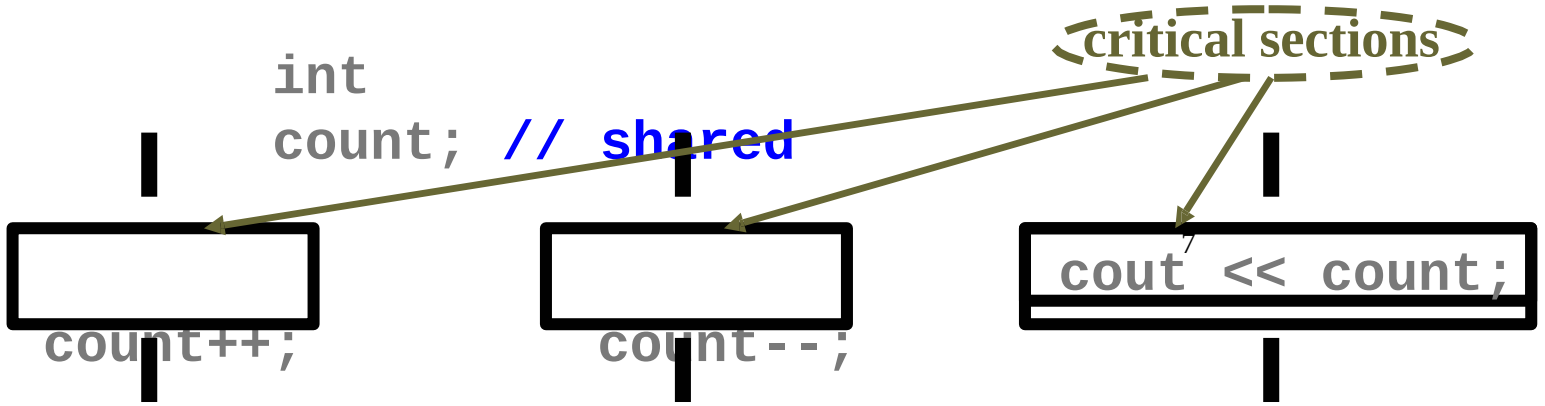


Race Conditions

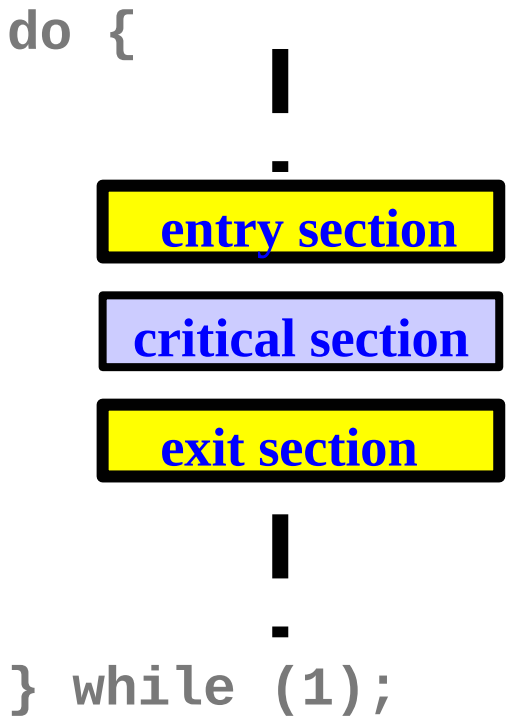
- **A *Race Condition* occurs, if**
  - **two or more processes/threads access and manipulate the same data concurrently, and**
  - **the outcome of the execution depends on the particular order in which the access takes place.**
- ***Synchronization* is needed to prevent race conditions from happening.**

Critical Section and Mutual Exclusion

- A *critical section* is a section of code in which a process accesses shared resources.
- Thus, the execution of critical sections must be *mutually exclusive* (*e.g.*, at most one process can be in its critical section at any time).
- The *critical-section problem* is to design a protocol that processes can use to cooperate.



The Critical Section Protocol



- A **critical section protocol** consists of **two** parts: an *entry section* and an *exit section*.
- Between them is the critical section that must run in a **mutually exclusive** way.

The Critical-Section Problem

- *n* processes competing to use some shared data.
- Each process has a code segment, called *Critical Section (CS)*, in which the shared data is accessed.
- Problem – ensure that when one process is executing in its CS, no other process is allowed to execute in its CS.

CS Problem Dynamics (1)

- When a process executes code that manipulates shared data (or resource), we say that the process is in it's Critical Section (for that shared data).
- The execution of critical sections must be mutually exclusive: at any time, only one process is allowed to execute in its critical section (even with multiple processors).
- So each process must first request permission to enter its critical section.

CS Problem Dynamics (2)

- The section of code implementing this request is called the Entry Section (ES).
- The critical section (CS) might be followed by a Leave/Exit Section (LS).
- The remaining code is the Remainder Section (RS).

Solution to Critical-Section Problem

- There are 3 requirements that must stand for a correct solution:
  1. **Mutual Exclusion**
  2. **Progress**
  3. **Bounded Waiting**



- We can check on all three requirements in each proposed solution, even though the non-existence of each one of them is enough for an incorrect solution.

Solution to CS Problem – Mutual Exclusion

1. **Mutual Exclusion** – If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.

- Implications:
  - Critical sections better be focused and short.
  - Better not get into an infinite loop in there.
  - If a process somehow halts/waits in its critical section, it must not interfere with other processes

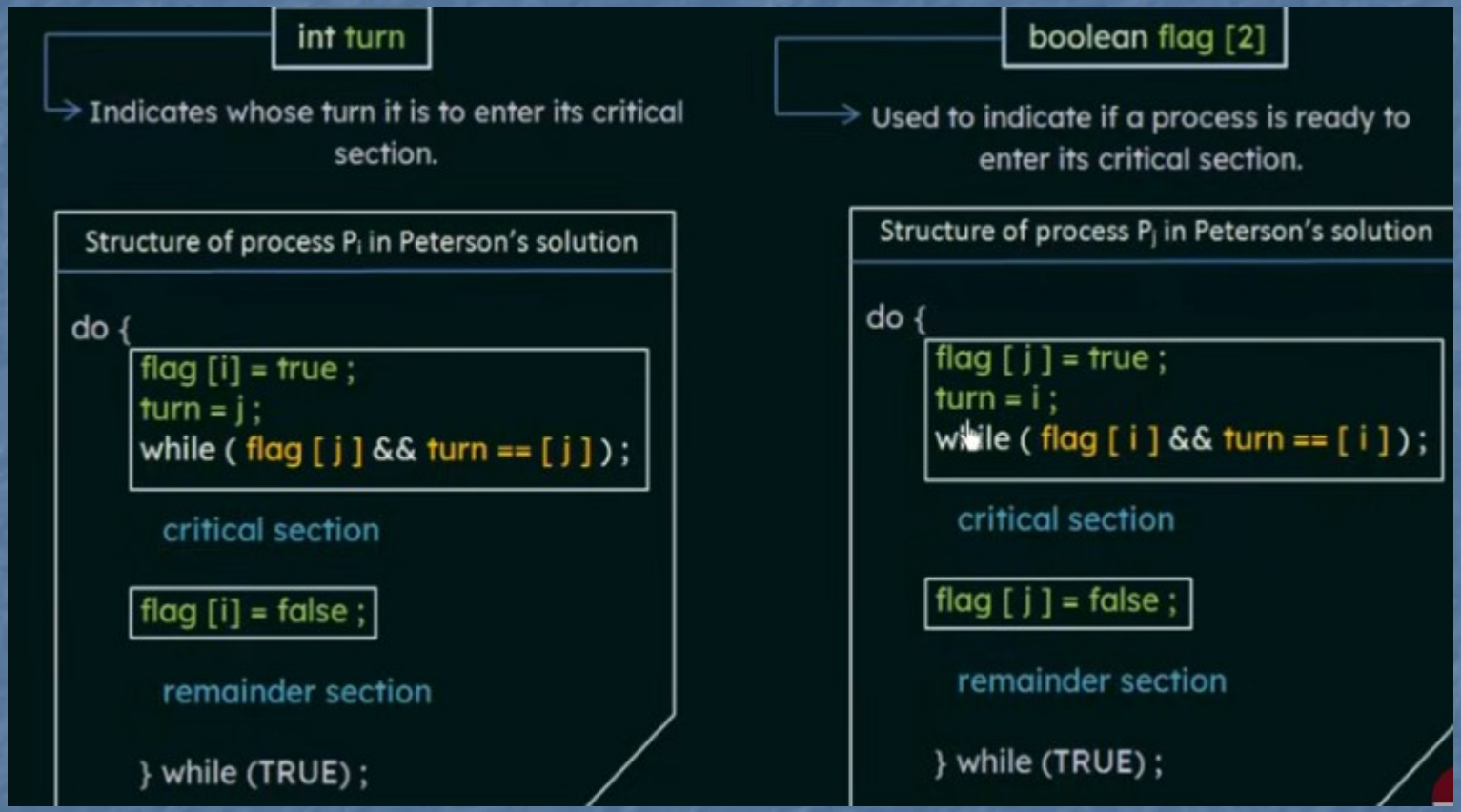
Solution to CS Problem – Progress

2. **Progress –** If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the process that will enter the critical section next cannot be postponed indefinitely:
   - If only one process wants to enter, it should be able to.
   - If two or more want to enter, one of them should succeed.

Solution to CS Problem – Bounded Waiting

3. **Bounded Waiting** – A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

   Hence, even though a process may be blocked by other waiting processes, it will not be waiting forever.


int turn
Indicates whose turn it is to enter its critical section.
boolean flag [2]
Used to indicate if a process is ready to enter its critical section.
Structure of process P_i in Peterson's solution
do {
flag [i] = true ;
turn = j ;
while (flag [j] && turn == [j]) ;
critical section
flag [i] = false ;
remainder section
} while (TRUE) ;
Structure of process P_j in Peterson's solution
do {
flag [j] = true ;
turn = i ;
while (flag [i] && turn == [i]) ;
critical section
flag [j] = false ;
remainder section
} while (TRUE) ;